

TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA CÔNG NGHỆ THÔNG TIN



Đề tài:

Phát triển hệ thống đèn LED thông minh RGB

với ESP32

Ghi chú: Điều khiển màu sắc và độ sáng của đèn LED WS2812 qua ứng dụng web hoặc giọng nói (kết hợp Google Assistant)

TÊN LỚP HỌC PHẦN : Phát triển ứng dụng IoT

MÃ HỌC PHẦN: 2025-2026.2.TIN4024.003

GIẢNG VIÊN HƯỚNG DẪN: Võ Việt Dũng

HUẾ, THÁNG 4 NĂM 2026

LỜI CẢM ƠN

Để hoàn thành bài tiểu luận này, trước tiên em xin gửi lời tri ân sâu sắc đến quý thầy cô khoa Công nghệ Thông tin, đặc biệt là thầy Võ Việt Dũng – giảng viên bộ môn Phát triển ứng dụng IoT. Sự hướng dẫn tận tình, lòng nhiệt huyết cùng những kiến thức nền tảng quý báu mà thầy truyền đạt chính là hành trang giúp em gắn kết lý thuyết với thực tiễn, từ đó tự tin hoàn thành đề tài nghiên cứu của mình.

Dù đã có nhiều cố gắng trong quá trình thực hiện, nhưng do kiến thức và kinh nghiệm còn hạn chế, bài tiểu luận chắc chắn không tránh khỏi những sai sót nhất định. Kính mong nhận được sự đánh giá và những lời góp ý chân thành từ quý thầy cô để em có cơ hội học hỏi và hoàn thiện hơn.

Em cũng xin gửi lời cảm ơn chân thành đến gia đình và bạn bè – điểm tựa tinh thần vững chắc, luôn động viên và tiếp thêm sức mạnh để em vượt qua mọi khó khăn trong học tập.

Một lần nữa, em xin chân thành cảm ơn thầy Võ Việt Dũng cùng tất cả những cá nhân đã hỗ trợ, đồng hành cùng em trong suốt thời gian qua.

MỤC LỤC

I. Tổng quan	4
1. Giới thiệu tổng quan	4
2. Mục tiêu của hệ thống	5
II. Nội dung	7
1. ESP32	7
2. LED WS2812 (Neopixel WS2812)	8
3. Cơ chế hoạt động	9
III. Thiết kế hệ thống	10
1. Phần cứng	10
2. Phần mềm	11
3. Cài đặt và lập trình	12
IV. Kết luận	16
1. Kết quả đạt được	16
2. Hạn chế và thách thức	16
3. Hướng phát triển	17
4. Kết luận	18
Tài liệu tham khảo	19

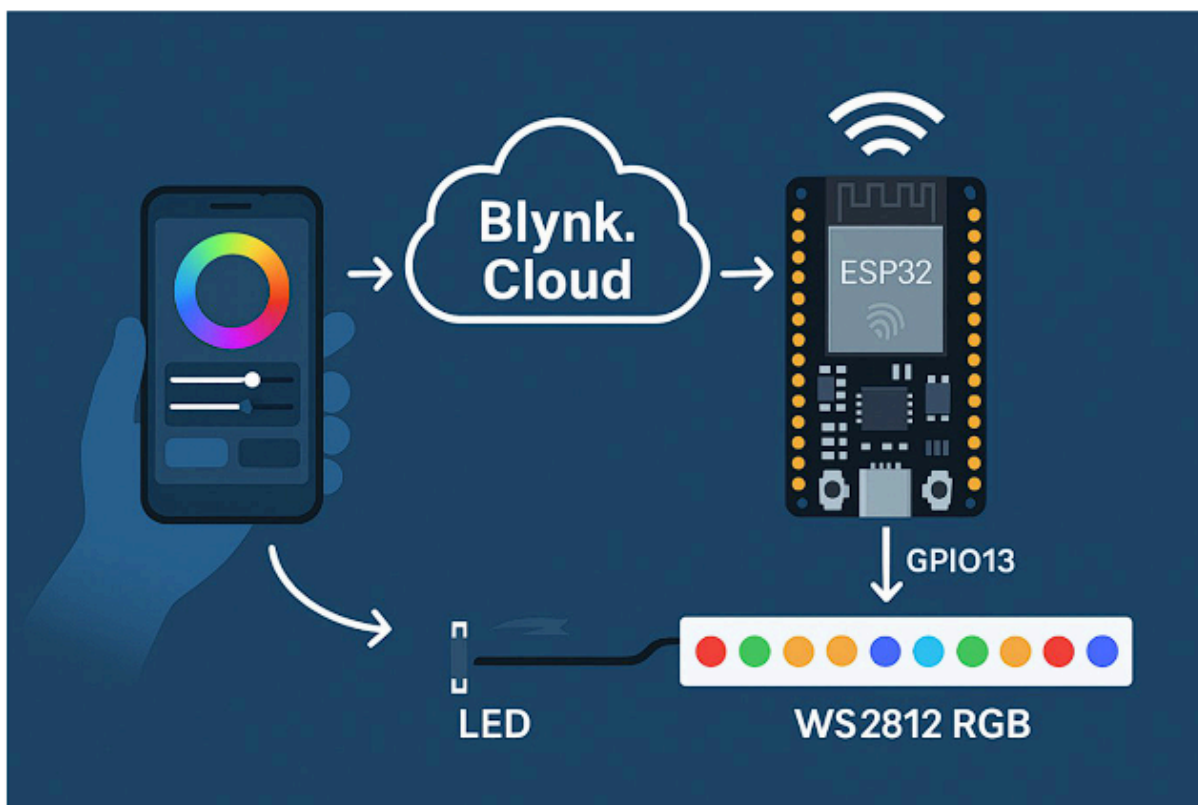
I. Tổng quan

1. Giới thiệu tổng quan

Trong kỷ nguyên Công nghiệp 4.0, hệ thống nhà thông minh (Smart Home) không chỉ là xu hướng mà đã trở thành tiêu chuẩn mới, góp phần nâng tầm chất lượng cuộc sống. Trong hệ sinh thái đó, chiếu sáng thông minh đóng vai trò cốt lõi. Việc ứng dụng công nghệ vạn vật kết nối (**IoT - Internet of Things**) vào các thiết bị chiếu sáng không chỉ tối ưu hóa trải nghiệm tiện nghi, tiết kiệm năng lượng mà còn đáp ứng các tiêu chuẩn khắt khe về thẩm mỹ kiến trúc kiến tạo không gian sống.

Xuất phát từ nhu cầu thực tiễn đó, dự án "Phát triển hệ thống đèn LED thông minh RGB sử dụng vi điều khiển **ESP32**" được nghiên cứu và thực hiện. Hệ thống mang đến một giải pháp chiếu sáng hiện đại, linh hoạt với khả năng điều khiển từ xa thông qua kết nối Internet. Trái tim của hệ thống là vi điều khiển trung tâm **ESP32**, kết hợp cùng dải LED thông minh **WS2812 (NeoPixel)** – cho phép định địa chỉ và tùy biến màu sắc, cường độ sáng của từng mắt LED độc lập chỉ với một đường truyền dữ liệu duy nhất (**One-wire protocol**) qua chân tín hiệu **GPIO5**.

Điểm nhấn công nghệ của dự án là khả năng vận hành đa nền tảng. Người dùng có thể dễ dàng giám sát và thay đổi các kịch bản ánh sáng trực tiếp trên giao diện Web Dashboard hoặc ứng dụng di động thông qua nền tảng đám mây **Blynk IoT**. Đồng thời, dự án cũng định hướng mở rộng khả năng điều khiển bằng giọng nói qua các trợ lý ảo thông minh (như Google Assistant). Toàn bộ quá trình phát triển được thực hiện bài bản bằng các công cụ chuyên nghiệp: lập trình C/C++ trên **Visual Studio Code**, mô phỏng mạch điện tử thực tế ảo trên nền tảng **Wokwi** và quản lý luồng dữ liệu đồng bộ qua **Blynk.Cloud**.



Hình ảnh mô tả hệ thống

2. Mục tiêu của hệ thống

Dự án được triển khai nhằm giải quyết các bài toán kỹ thuật và ứng dụng thực tiễn với những mục tiêu trọng tâm sau:

Làm chủ công nghệ phần cứng và giao thức điều khiển: Thiết kế và triển khai thành công hệ thống mạch điều khiển dải đèn LED WS2812 bằng vi điều khiển ESP32. Đảm bảo khả năng xử lý tín hiệu mượt mà, tùy biến màu sắc chuẩn xác theo dải màu RGB và tinh chỉnh cường độ sáng theo thời gian thực.

Phát triển giao diện đa nền tảng (UI/UX) tối ưu: Xây dựng bảng điều khiển trực quan, thân thiện với người dùng trên cả hai nền tảng Web và thiết bị di động (Mobile App) thông qua hệ sinh thái Blynk.Cloud, giúp người dùng làm chủ hệ thống chiếu sáng ở mọi lúc, mọi nơi không giới hạn khoảng cách địa lý.

Tiên phong trải nghiệm và mở rộng tính năng: Tích hợp tính năng nhận diện lệnh điều khiển bằng giọng nói (Voice Control) thông qua Google Assistant hoặc các nền tảng tương đương, qua đó nâng cao mức độ rảnh tay và mang lại trải nghiệm không gian sống đậm chất tương lai.

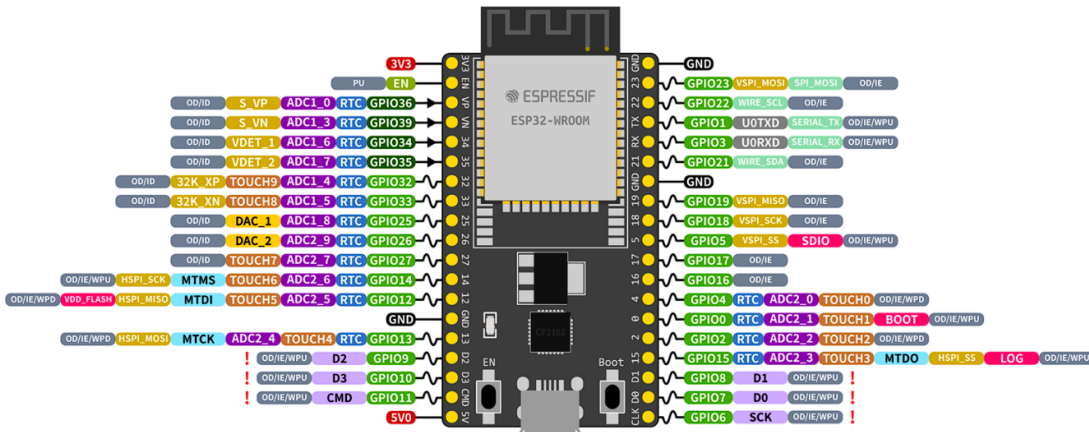
Tối ưu hóa hiệu năng và độ ổn định: Cải tiến mã nguồn (Source Code) và cơ sở hạ tầng mạng để hệ thống vận hành bền bỉ, phản hồi tín hiệu tức thời với độ trễ thấp nhất. Đảm bảo sản phẩm có khả năng mở rộng quy mô (scale-up) để ứng dụng trực tiếp vào các lĩnh vực như: chiếu sáng kiến trúc, trang trí nội thất không gian làm việc hoặc nghệ thuật trình diễn ánh sáng.

Xây dựng nền tảng nghiên cứu IoT: Tạo ra một bộ khung (framework) kỹ thuật chuẩn mực, làm tiền đề vững chắc cho việc tiếp tục nghiên cứu, phát triển và tích hợp thêm các cảm biến thông minh khác vào hệ sinh thái Smart Home.

II. Nội dung

1.ESP32

ESP32-DevKitC



Hình ảnh minh họa ESP32

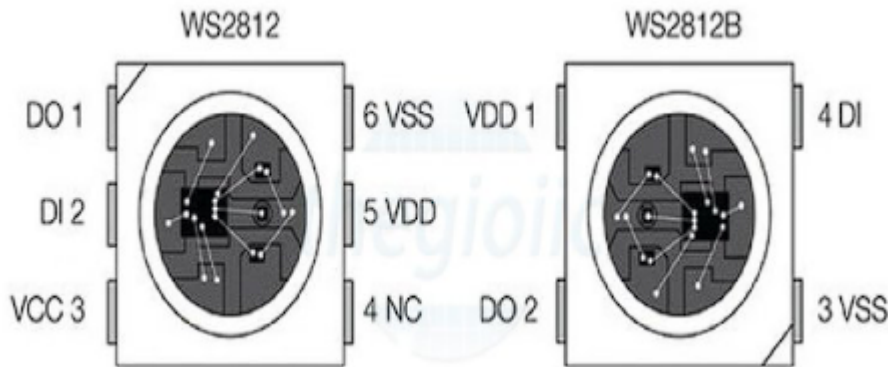
ESP32 là một hệ thống vi điều khiển trên chip (SoC) chi phí thấp, hiệu năng cao do Espressif Systems phát triển. Là phiên bản kế nhiệm của ESP8266, ESP32 được trang bị bộ vi xử lý 32-bit Xtensa LX6 (có cả biến thể lõi đơn và lõi kép), tích hợp sẵn chuẩn giao tiếp không dây Wi-Fi và Bluetooth. Được sản xuất trên tiến trình công nghệ 40nm siêu tiết kiệm năng lượng của TSMC, ESP32 trở thành lựa chọn lý tưởng cho các ứng dụng IoT, thiết bị đeo thông minh và hệ thống nhà thông minh.

Thông số kỹ thuật nổi bật:

- Xung nhịp: Lên đến 240 MHz.
- Bộ nhớ: 520 KB SRAM, 448 KB ROM và 16 KB SRAM RTC.
- Kết nối mạng: Hỗ trợ Wi-Fi chuẩn 802.11 b/g/n với tốc độ truyền tải đạt 150 Mbps.
- Kết nối tiệm cận: Hỗ trợ Bluetooth v4.2 và Bluetooth Low Energy (BLE).
- Ngoại vi: Tích hợp 34 chân GPIO có thể lập trình, 18 kênh ADC 12-bit, 2 kênh DAC 8-bit và 16 kênh PWM.
- Giao thức nối tiếp: Hỗ trợ đa dạng gồm 4x SPI, 2x I2C, 2x I2S, 3x UART.

- Bảo mật: Tích hợp bộ tăng tốc phần cứng mã hóa cho AES, Hash (SHA-2), RSA, ECC và bộ tạo số ngẫu nhiên (RNG).

2.LED WS2812(Neopixel WS2812)



Hình ảnh minh họa Neopixel WS2812

WS2812 là một mạch tích hợp bao gồm 3 bóng LED cơ bản (Đỏ, Xanh lá, Xanh dương) và 1 IC điều khiển được đóng gói trong cùng một module duy nhất. Điểm đột phá của dòng LED này là khả năng kết nối nối tiếp nhiều bóng với nhau (lên tới 144 bóng riêng biệt) nhưng chỉ yêu cầu duy nhất 1 đường tín hiệu điều khiển (Giao thức One-wire).

Mỗi IC điều khiển sở hữu một thanh ghi 24-bit (chia làm 3 phần, mỗi phần 8-bit tương ứng cho dải màu R-G-B), cho phép điều chỉnh 256 mức độ sáng cho mỗi dải màu. Sự kết hợp này tạo ra khả năng hiển thị lên tới xấp xỉ 17 triệu màu sắc khác nhau trên mỗi mắt LED.

Cấu tạo chân giao tiếp và thông số:

- VDD / VCC: Chân cấp nguồn cho LED và IC (Điện áp hoạt động từ 3.5V đến 5.3V).
- GND: Chân nối đất (Mass).
- DIN: Chân nhận dữ liệu tín hiệu đầu vào.
- DOUT: Chân xuất dữ liệu tín hiệu để truyền sang bóng LED kế tiếp.
- Góc chiếu sáng: 120 độ.

3. Cơ chế hoạt động

Hệ thống điều khiển LED RGB thông minh hoạt động dựa trên sự phối hợp đồng bộ giữa phần mềm quản lý đám mây (Blynk.Cloud) và vi điều khiển phần cứng (ESP32) thông qua 5 bước chính:

- Bước 1: Khởi tạo và thiết lập hệ thống: Sau khi được cấp nguồn, ESP32 sẽ tự động kết nối với mạng Wi-Fi đã được lập trình sẵn. Tiếp đó, thông qua hàm **Blynk.begin()**, hệ thống sử dụng mã định danh Auth Token để xác thực và thiết lập luồng giao tiếp bảo mật với máy chủ Blynk.Cloud.
- Bước 2: Lắng nghe tín hiệu (Giao tiếp người dùng): Người dùng thao tác trên ứng dụng Blynk (Web hoặc Mobile) thông qua các tiện ích (Widget) như Text Input (gắn với chân ảo V0) và Thanh trượt độ sáng - Slider (gắn với chân ảo V1).
- Bước 3: Truyền tải và phân tích dữ liệu: Khi có sự thay đổi từ UI, lệnh điều khiển lập tức được truyền qua Internet tới máy chủ Blynk và đẩy thẳng xuống ESP32. ESP32 tiến hành thu nhận mã màu dưới định dạng HEX (Ví dụ: **#FF0000**) và mức độ sáng (0-255). Chương trình sẽ tự động nội suy và bóc tách dải mã HEX này thành 3 giá trị màu cơ bản (Red, Green, Blue) kết hợp với hệ số độ sáng.
- Bước 4: Xuất tín hiệu điều khiển: Vi điều khiển sử dụng thư viện **Adafruit_NeoPixel** để biên dịch dữ liệu và xuất chuỗi tín hiệu ra chân GPIO5 (chân này được nối với chân DIN của dải LED WS2812 thông qua một điện trở bảo vệ 330Ω). Tín hiệu được truyền nối tiếp theo kiểu "dịch vòng", bóng LED đầu tiên nhận dữ liệu của nó và đẩy phần dữ liệu còn lại sang bóng kế tiếp, tạo thành một chuỗi đồng bộ.
- Bước 5: Đáp ứng thời gian thực: Ngay khi nhận đủ chuỗi tín hiệu, toàn bộ dải LED WS2812 sẽ cập nhật và hiển thị chính xác màu sắc cùng cường độ sáng mà người dùng thiết lập. Quá trình này diễn ra liên tục với độ trễ cực thấp (gần như thời gian thực), hệ thống luôn trong trạng thái sẵn sàng nhận lệnh mới mà không cần khởi động lại.

III. Thiết kế hệ thống

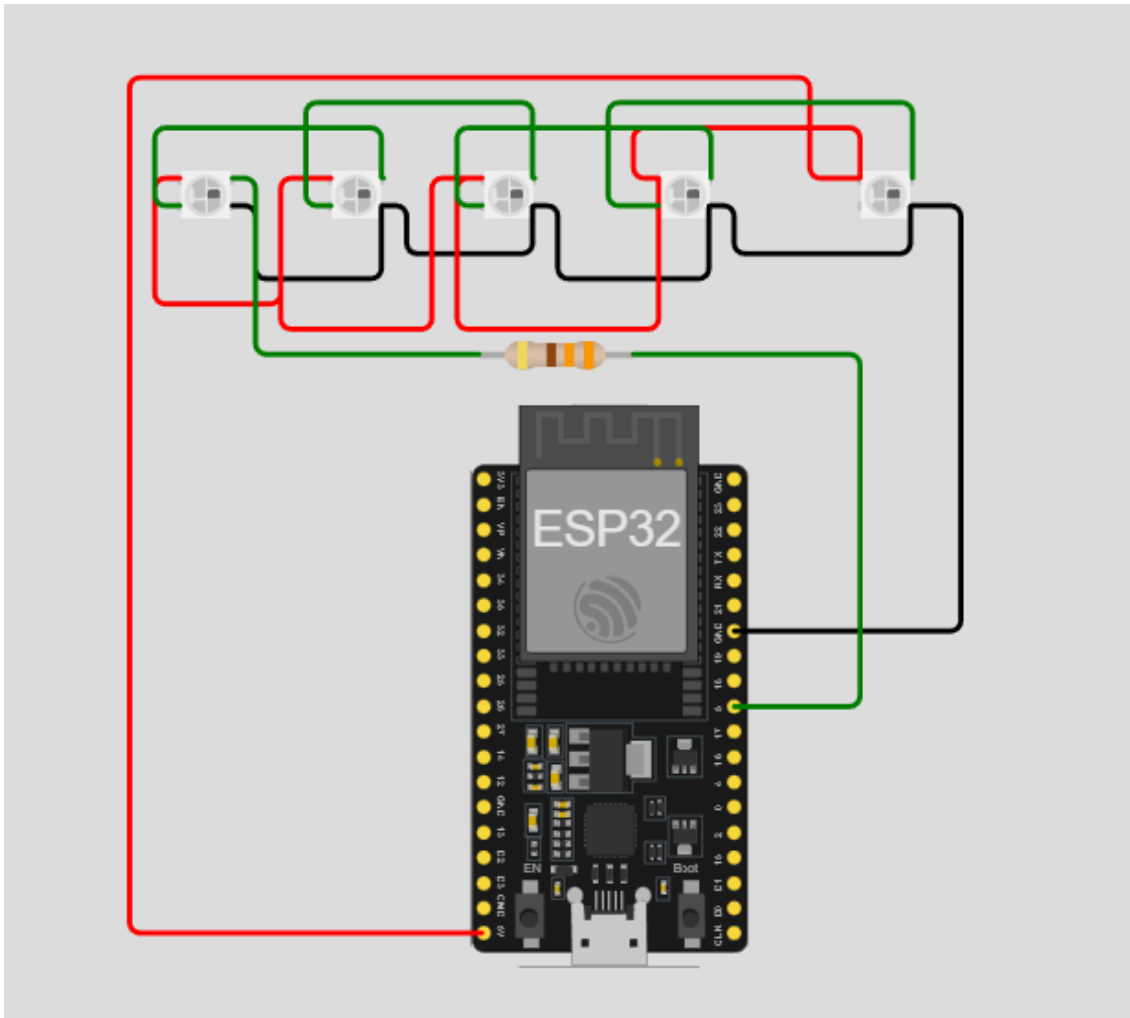
1. Phần cứng

Hệ thống được thiết kế trên nền tảng vi điều khiển trung tâm ESP32 kết hợp với các linh kiện sau:

- ESP32 Devkit: Vi điều khiển chính, hỗ trợ kết nối Wi-Fi và chịu trách nhiệm xử lý tín hiệu, điều khiển LED thông qua xung PWM.
- Dải LED WS2812 (NeoPixel): Bao gồm các mắt bóng LED RGB có thể lập trình và định địa chỉ độc lập.
- Điện trở 330Ω: Được kết nối nối tiếp giữa chân tín hiệu của ESP32 (GPIO5) và chân DATA (DIN) của LED WS2812, đóng vai trò bảo vệ mạch và giảm nhiễu tín hiệu truyền tải.
- Nguồn điện 5V: Cung cấp năng lượng hoạt động cho dải LED (có thể trích xuất từ chân 5V/VIN của ESP32 hoặc sử dụng nguồn tổ ong/adapter ngoài nếu dải LED quá dài để đảm bảo đủ dòng).

Sơ đồ kết nối cơ bản:

- Chân GPIO5 (ESP32) —> Điện trở 330Ω —> Chân DIN (WS2812)
- Chân GND (ESP32) —> Chân GND (WS2812)
- Chân 5V/VIN (ESP32) —> Chân VCC/5V (WS2812)



Hình ảnh sơ đồ mạch điện mô phỏng giữa ESP32 và Neopixel WS2812

2. Phần mềm

Các công cụ và nền tảng phần mềm được sử dụng để phát triển dự án bao gồm:

- **Visual Studio Code (PlatformIO):** Môi trường phát triển tích hợp (IDE) chuyên nghiệp, được sử dụng để soạn thảo, biên dịch mã nguồn (C/C++) và nạp chương trình trực tiếp vào ESP32.
- **Thư viện BlynkSimpleEsp32.h:** Đóng vai trò cầu nối, hỗ trợ ESP32 kết nối với nền tảng đám mây Blynk.Cloud. Nhờ đó, vi điều khiển có thể nhận lệnh điều khiển từ xa thông qua Internet.
- **Thư viện WiFi.h:** Thư viện tiêu chuẩn giúp ESP32 thiết lập kết nối với mạng Wi-Fi cục bộ, điều kiện bắt buộc để giao tiếp với máy chủ Blynk.
- **Thư viện Adafruit_NeoPixel.h:** Thư viện chuyên dụng để điều khiển dải LED WS2812, hỗ trợ đóng gói và truyền tải dữ liệu điều khiển phức tạp qua một chân GPIO duy nhất (Giao thức One-wire), giúp việc lập trình

hiệu ứng ánh sáng trở nên đơn giản và tối ưu hơn.

- **Nền tảng Blynk.Cloud (Web/App):** Hệ sinh thái IoT cho phép khởi tạo nhanh giao diện người dùng (UI) trực quan. Thông qua các Widget trên Blynk, người dùng có thể điều khiển hệ thống chiếu sáng từ bất cứ đâu có kết nối Internet.

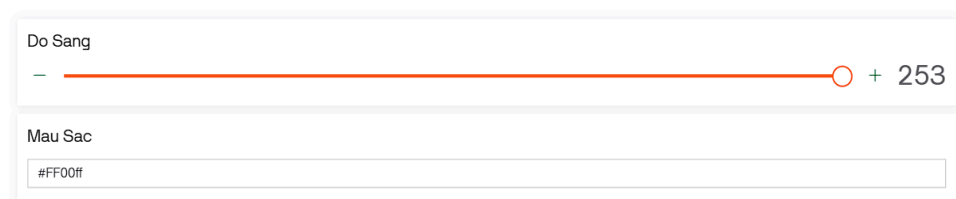
3. Cài đặt và lập trình

- **Tạo tài khoản và Template trên Blynk.Cloud:** Truy cập hệ thống Blynk, khởi tạo một Template mới cho dự án. Hệ thống sẽ cấp một chuỗi **Auth Token** duy nhất. Mã này, cùng với ID và tên Template, sẽ được khai báo ở đầu mã nguồn (Source Code) trên VS Code để xác thực thiết bị.

```
#define BLYNK_TEMPLATE_ID "TMPL6pK8E6c7E"  
#define BLYNK_TEMPLATE_NAME "LED RGB SMART"  
#define BLYNK_AUTH_TOKEN "O_0jfBD80GMVLDCr_N_ELnwNaKHvCjtK"
```

Hình ảnh minh họa cấu hình kết nối với Blynk

- **Cấu hình giao diện (Dashboard):** Trên ứng dụng Blynk (Mobile/Web), tạo giao diện điều khiển bằng cách thêm các Widget. Điểm nhấn của hệ thống là sử dụng tiện ích **Text Input** (gán với chân ảo V0) để người dùng có thể nhập trực tiếp các mã màu chuẩn xác (ví dụ: mã HEX **#FF00ff** cho màu đỏ), kết hợp cùng tiện ích **Slider** (gán với chân ảo V1) để tinh chỉnh cường độ sáng của đèn.



Hình ảnh minh họa hệ thống Blynk

Lập trình Visual Studio Code với PlatformIO IDE: Cài đặt thư viện Blynk và Adafruit NeoPixel từ Library Manager. Tạo project mới, khai báo thông tin Wi-Fi, Auth Token và thiết lập các chân kết nối LED.

```
LED_RGB > src > main.cpp > ...
You, 1 minute ago | 1 author (You)
1 #define BLYNK_TEMPLATE_ID "TMPL6pK8E6c7E"
2 #define BLYNK_TEMPLATE_NAME "LED_RGB SMART"
3 #define BLYNK_AUTH_TOKEN "O_0jfbD80GMVLDcr_N_ElnwNakHvCjtk"
4
5 #include <WiFi.h>
6 #include <WiFiClient.h>
7 #include <BlynkSimpleEsp32.h>
8 #include <Adafruit_NeoPixel.h>
9 | You, 4 days ago * pbhoa_BAITHI_LED-rgb_WS2812 ---
10 // --- Cấu hình WiFi cho mô phỏng Wokwi ---
11 char ssid[] = "Wokwi-GUEST";
12 char pass[] = "";
13
14 // --- Cấu hình LED ---
15 #define PIN 5
16 #define NUMPIXELS 5
17 Adafruit_NeoPixel pixels(NUMPIXELS, PIN, NEO_GRB + NEO_KHZ800);
18
19 // Biến lưu trạng thái hiện tại (Mặc định sáng màu trắng)
20 int r = 255, g = 255, b = 255;
21 int currentBrightness = 255;
22
23 // --- HÀM CẬP NHẬT ĐÈN (Phải đặt ở đây để các hàm bên dưới gọi được) ---
24 void updateLEDs() {
25     for(int i=0; i<NUMPIXELS; i++) {
26         // Tính toán lại màu dựa trên % độ sáng
27         pixels.setPixelColor(i, pixels.Color(r * currentBrightness / 255, g * currentBrightness / 255, b * currentBrightness / 255));
28     }
29     pixels.show(); // Hiển thị màu sắc mới trên LED
30 }
31
32 // --- BLYNK: Nhận màu từ color picker (Virtual pin V0) ---
33 BLYNK_WRITE(V0) {
34     String color = param.asStr(); // Lấy giá trị màu từ color picker
35     long number = strtoul(&color[1], NULL, 16); // Chuyển đổi từ hex string sang số nguyên
36     r = (number >> 16) & 0xFF; // Lấy giá trị đỏ
37     g = (number >> 8) & 0xFF; // Lấy giá trị xanh lá
38     b = number & 0xFF; // Lấy giá trị xanh dương
39     updateLEDs(); // Cập nhật màu sắc cho LED
40 }
41
```

```
41
42 // --- BLYNK: Nhận giá trị độ sáng từ slider (Virtual pin V1) ---
43 BLYNK_WRITE(V1) {
44     currentBrightness = param.asInt(); // Lấy giá trị độ sáng từ slider
45     updateLEDs(); // Cập nhật màu sắc cho LED
46 }
47
48 void setup() {
49     Serial.begin(115200);
50     pixels.begin(); // Khởi tạo thư viện NeoPixel
51     updateLEDs(); // Cho đèn sáng ngay lúc vừa khởi động
52
53     Serial.println("Đang kết nối đến Blynk...");
54     Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
55 }
56
57 // --- HÀM LOOP BẮT BUỘC PHẢI CÓ ĐỂ DUY TRÌ BLYNK ---
58 void loop() {
59     Blynk.run();
60 }
```

Hình ảnh minh họa mã nguồn main.cpp

***Quy trình hoạt động của chương trình:**

Chương trình điều khiển LED RGB thông minh hoạt động theo chuỗi 5 bước logic rõ ràng, kết hợp nhuần nhuyễn giữa thao tác người dùng và khả năng đáp ứng của phần cứng:

- **Bước 1: Khởi tạo hệ thống**
 - ESP32 được cấp nguồn và khởi động chương trình đã nạp.
 - Hệ thống tự động kết nối với mạng Wi-Fi thông qua thông tin SSID và Password đã được lập trình sẵn.
 - Thông qua hàm **Blynk.begin()**, ESP32 kết nối với máy chủ Blynk.Cloud và định danh thiết bị bằng mã Auth Token.
- **Bước 2: Lắng nghe tín hiệu điều khiển từ Blynk**
 - Thông qua giao diện Blynk, người dùng tương tác bằng cách nhập chuỗi mã màu mong muốn vào widget **Text Input** (chân ảo V0) và kéo thanh trượt **Slider** (chân ảo V1) để thay đổi độ sáng.
 - Các widget này lập tức đóng gói dữ liệu và gửi lên máy chủ Blynk, sau đó chuyển tiếp thẳng đến vi điều khiển ESP32 qua Internet.
- **Bước 3: Nhận và xử lý dữ liệu tại ESP32**
 - ESP32 tiếp nhận dữ liệu chuỗi văn bản (String) từ chân V0 (ví dụ: **#FF0000**) và giá trị số nguyên từ chân V1 (dải từ 0-255 cho độ sáng).
 - Thuật toán trong mã nguồn sẽ tiến hành phân tích (parse) chuỗi ký tự vừa nhập, bóc tách nó thành 3 giá trị màu cơ bản (Red, Green, Blue).
 - Các giá trị RGB này tiếp tục được tính toán kết hợp với hệ số từ thanh trượt độ sáng để cho ra thông số hiển thị cuối cùng.
- **Bước 4: Điều khiển LED WS2812**
 - Vi điều khiển gọi các hàm từ thư viện **Adafruit_NeoPixel** để biên dịch các thông số trên thành chuỗi tín hiệu kỹ thuật số.
 - Dữ liệu được đẩy ra ngoài qua chân GPIO5 (đã được nối tiếp qua điện trở 330Ω để giảm nhiễu).

- Tín hiệu truyền nối tiếp đến chân DIN của dải LED. Mỗi mắt LED sẽ trích xuất phần dữ liệu của riêng mình và đẩy phần còn lại cho mắt LED kế tiếp, tạo thành một chuỗi đồng bộ.
- **Bước 5: Hiển thị màu sắc**
 - Dải LED ngay lập tức phát sáng với chuẩn màu và cường độ sáng mà người dùng đã nhập.
 - Quá trình thay đổi trạng thái này diễn ra với tốc độ rất cao (gần như thời gian thực). Hệ thống tiếp tục vòng lặp lắng nghe, sẵn sàng nhận các chuỗi lệnh mới và cập nhật liên tục mà không yêu cầu khởi động lại thiết bị.

IV. Kết luận

1. Kết quả đạt được

Sau quá trình thiết kế, lập trình và tinh chỉnh, dự án đã được triển khai mô phỏng thành công trên nền tảng Wokwi và đáp ứng đầy đủ các tiêu chí kỹ thuật đã đề ra:

- **Khởi chạy và kết nối ổn định:** Hệ thống vi điều khiển ESP32 và dải LED WS2812 hoạt động trơn tru trong môi trường mô phỏng. Thiết bị đã tự động thiết lập kết nối Wi-Fi và đồng bộ hóa thành công với máy chủ đám mây Blynk.Cloud thông qua mã xác thực Auth Token.
- **Vận hành giao diện chính xác:** Bảng điều khiển (Dashboard) hoạt động đúng thiết kế. Widget **Text Input** ghi nhận và truyền tải chính xác chuỗi mã màu (HEX) do người dùng nhập, trong khi widget **Slider** kiểm soát tốt luồng dữ liệu số nguyên (0-255) cho cường độ sáng.
- **Đáp ứng thời gian thực (Real-time response):** Thuật toán phân tích mã nguồn hoạt động hiệu quả. Khi người dùng nhập một mã màu bất kỳ (Ví dụ: **#FF0000**), hệ thống lập tức bóc tách thành dải RGB và xuất tín hiệu điều khiển, dải đèn LED phản hồi chuyển sang màu Đỏ ngay lập tức với độ trễ (latency) không đáng kể. Thao tác vuốt thanh trượt cũng làm thay đổi độ sáng đèn một cách tuyến tính và mượt mà.

2. Hạn chế và thách thức

Từ các kết quả thử nghiệm, hệ thống bộc lộ những ưu điểm và một số hạn chế nhất định cần lưu ý:

- **Ưu điểm vượt trội:**
 - **Tối ưu chi phí và rủi ro:** Việc sử dụng nền tảng Wokwi cho phép kiểm thử toàn diện logic lập trình mà không lo ngại rủi ro chập cháy hay tốn kém chi phí mua sắm linh kiện phần cứng ở giai đoạn đầu.
 - **Tính linh hoạt và cá nhân hóa cao:** Giao diện điều khiển UI/UX có thể được tùy biến dễ dàng. Người dùng toàn quyền quyết định màu sắc hiển thị không giới hạn thông qua bảng mã HEX.

- **Khả năng điều khiển phi khoảng cách:** Nhờ kiến trúc IoT và nền tảng Blynk, hệ thống xóa bỏ rào cản về khoảng cách địa lý, cho phép quản lý không gian ánh sáng từ bất cứ đâu miễn là có Internet.
- **Hạn chế tồn đọng:**
 - **Phụ thuộc vào hạ tầng mạng:** Do bản chất của một hệ thống Cloud IoT, thiết bị yêu cầu kết nối Wi-Fi và Internet liên tục. Nếu đường truyền mạng gián đoạn, khả năng điều khiển từ xa sẽ bị vô hiệu hóa.
 - **Độ sai lệch vật lý:** Môi trường Wokwi là lý tưởng. Khi thi công trên phần cứng thực tế, hệ thống có thể đối mặt với các vấn đề vật lý như: sụt áp ở cuối dải LED nếu dải quá dài, nhiễu tín hiệu đường truyền, hoặc sai lệch màu sắc thực tế do nhiệt độ linh kiện.

3. Hướng phát triển

Dựa trên nền tảng kỹ thuật đã hoàn thiện, dự án mở ra nhiều tiềm năng nâng cấp trong tương lai:

- **Hiện thực hóa phần cứng:** Triển khai lắp ráp mạch điện trên các linh kiện vật lý thực tế, thiết kế hộp bảo vệ (case 3D) và thi công lắp đặt dải LED vào không gian nội thất.
- **Tích hợp Trợ lý ảo (Voice Control):** Liên kết máy chủ Blynk với các dịch vụ Webhook/IFTTT để đưa hệ thống vào hệ sinh thái Google Home hoặc Amazon Alexa, cho phép người dùng ra lệnh đổi màu đèn bằng giọng nói.
- **Phát triển tính năng tự động hóa (Automation):** Bổ sung các cảm biến môi trường (Cảm biến ánh sáng LDR, cảm biến âm thanh) để đèn có khả năng tự động bật/tắt theo cường độ sáng xung quanh hoặc chớp nháy đồng bộ theo nhịp điệu âm nhạc.

4. Kết luận

Dự án "**Phát triển hệ thống đèn LED thông minh RGB sử dụng vi điều khiển ESP32**" đã hoàn thành xuất sắc mục tiêu cốt lõi: Xây dựng một giải pháp chiếu sáng thông minh, có thể điều khiển màu sắc và độ sáng linh hoạt từ xa thông qua nền tảng IoT. Dự án không chỉ chứng minh tính khả thi của việc ứng dụng vi điều khiển ESP32 trong lĩnh vực Smart Home, mà còn cung cấp một nền tảng kiến thức vững chắc về giao thức truyền tải dữ liệu, xử lý chuỗi tín hiệu và thiết kế giao diện tương tác người máy (HMI)

Tài liệu tham khảo

<https://di-ntutu-ngl.iic.m/sp32-l-gi.html>

<https://www.itzn.vn/sp32/sp32-c-bn/sp32-dc-dc-gi-tri-nlg/>

<https://www.thgiic.cm/tin-tuc/gi-i-thi-u-v-n-pix-l-ws2812>

<http://rduin.vn/tut-ri-l/1570-gi-i-thi-u-m-dul-sp32-v-hu-ng-d-n-c-i-trinh-bi-n-dich-tr-n-rduin-id>

<http://rduin.vn/bi-vi-t/1006-fi-t-lux-h-y-c-nh-s-ng-ph-n-2-vi-t-c-d-blink-th-nh-ch-n-pix-l-ws2812>